Kapil Tamang
Senior Project

B05 — Resilient Case Management Services

What problem are you investigating?

The Case Management System captures issues, assigns these to appropriate staff members and monitors progress until resolution of the issue. The problem in question is that of the integrity of case-related data in situations when the two services attempt to talk to each other and one goes down. In such cases, the case may be saved in one service, while the assignment, task, or notification connected with this case is not saved in another service. The situation is complicated by the fact that, in such cases, the link may go down between completion of an action in one service and the acknowledgment of this action by the other service.

Let's take that of a university student reporting a faulty projector. Under normal conditions, the student will report the fault, the administrator will assign it to the maintenance crew, and the case status will be updated from Pending to In Progress and then to Resolved. This is what both the student and the maintenance crew need to see. Let us now assume that an administrator makes the case In Progress. This will be recorded in the database, but the worker responsible for making the record of the assignment notification will stop. A simple design will result in the loss of this notification. Another problem will arise when the request times out even after being successful; this can lead to duplicate assignments.

This research is particularly focused on temporary service disruptions, interruption of communication, multiple requests and inconsistent or incomplete processing of case updates. This research is not concerned with whether the universities need complaint websites or what particular solution should be adopted. The main idea is to identify whether standard case management processes are still valid while operations span across services. Case queues are described by Microsoft as a tool to organize, prioritize, assign and monitor tasks; however, it does not measure failure rates.

Who or what is affected by it?

Immediate users include students or clients who submit cases, employees who perform the task, and administrators handling the assignments. A student can view the old status and submit a new complaint. The maintenance worker may never receive a new assignment, while another person will be assigned a duplicate assignment and end up performing unnecessary tasks. The administrator can generate a report indicating that the case remains unresolved when in fact it has been resolved. This is important since case data affects decision-making, and not just what is viewed on the screen.

The organization relies on accurate records for accountability purposes – who received the case, when was its status updated, and did the required follow-up take place. Should the

case data be inconsistent, employees will have to spend extra time resolving this issue manually. Technical components are also affected. The database, the case API, and the standalone background worker are all functioning well at certain points in time, leaving behind an unfinished process. The scope is not looking for the components to always stay online; instead, it is asking if the system knows how to deal with the unfinished job.

What evidence do you currently have that it is a real problem?

The posted course catalog has listed project B05, titled "Resilient Case Management Services" as one of the projects being offered. This demonstrates the fact that the idea of resilience in case management can serve as a good candidate topic for this course; however, the title itself is not sufficient to conclude that a particular organization has missed out on an update. What I require now is the full published opportunity with its official address.

Evidence from a technical perspective exists to support the underlying problem that causes the issue. In Microsoft's architectural guidance, there is information about how difficult it is to store business data and send an event in two distinct actions. In the Transactional Outbox pattern by Microsoft, it is mentioned that both the business data and the corresponding event are stored first, and then a publisher sends the event separately. There are other issues discussed like order preservation of the events.

The following piece of evidence about the uncertainty in completing a task is taken from Camunda's documentation independently. A worker could have done the task, but without completing it, as per the time of expiration of the task. This means that the task can be assigned once more, hence the process of doing the task could have occurred twice. Camunda demands for idempotence on the side of the worker.

Evidence Status Matters.Documented: Work routing and tracking by case systems and distribution and retries in messaging may cause problems in delivery or duplication.Reported: The course catalogue notes that resilience is a potential problem. Observed: I have neither prototyped nor interviewed any case workers, gathered any logs, or observed any incident.Hypothesized: A small-scale university-like case system running with independent services may show these failure modes and recovery rules may help increase the correctness of this system. These statements should not be used as measurement findings.

What solutions already exist?

Two helpful existing techniques are employed in solving different facets of the issue. The first technique is known as the Transactional Outbox pattern, which has been documented by Microsoft. The application stores the case change and the message to be sent as part of a single database transaction. Another operation is scheduled to deliver the message at some point. It makes it less likely that the committed case change will have no corresponding message due to any interruptions. Its advantages include clear roles and reliable pending messages. Its disadvantages involve maintaining the order of messages, recognizing duplications, and avoiding doing the same action twice.

The second approach is workflow coordination with the use of Camunda 8. Workflow engine manages tasks and supplies them to workers; workers who are not available need not to

be the only holders of incomplete tasks. Timeouts help incomplete tasks be available for other workers, and retries help continue from where they stopped. This method is important in situations where a case goes through multiple stages of review and approvals. However, its cons are complex operations and need for safe retrying. A workflow engine doesn't guarantee that every side effect happens only once.

A simpler baseline also deserves consideration: the case API can commit a database update and then call a worker directly. It is straightforward to understand and may be adequate for low-consequence follow-up actions, but its failure gap is precisely what needs examination. My comparison is not intended to show that existing tools are bad. It is to determine when a simpler approach is sufficient and when durable coordination becomes important. The assigned reading "Research and Analysis of Existing Solutions" argues for investigating existing technologies, strengths, and weaknesses before starting design.

What is the most important thing you still do not know?

The unknown that needs addressing is the conditions under which an update and activities will be skipped or retried, and if there is any condition under which using the lighter-weight approach of persistent event is better compared to the more complex approach of making the call directly. The documentation does provide information on the potential failure conditions, but it will not give me any insight into the frequency and implications of such failure conditions in my prototype application, which is not yet tested. What do I mean by "correct" anyway?

One further research stage would be to carry out a controlled experiment with fictional cases. Using either the direct call baseline approach or persistent event approach, I could make similar interruptions such as stopping a worker from completing an update, stopping it after the action but before acknowledgment, and repeating the same request. I would measure the number of missing actions, duplicate results, inaccurate state of the case and time taken to restart. This could start by conducting 100 updates in each scenario, and so on.

Two further open questions are how frequently the case management operators actually encounter outdated information or duplicates, and whether the B05 requirement itself has more stringent requirements than this example. The former can be answered by analyzing publicly available incident reports, or perhaps through interviews if permitted; for the latter, I will need the catalog entry. My answers will determine whether I should look at update resilience, safe handling of duplicates, or something else entirely.

Your Individual Scope and Senior Project Relationship

In order to explore the idea in more depth during this early investigation phase, I will be looking at a limited approach to creating cases and following up with them in a manner similar to university procedures: creating a case, assigning it, changing the case's status, and having a single background follow-up process happen. I will not be looking at the incorporation of actual student data, the university environment, legal implications, user interface development, cloud-level availability, or the complete destruction of the main database.

According to the engineering-design definitions from the first required reading, the probable assignment involves adaptive design, which means using proven reliability methods in

a new work flow, and redesign, which means improving the behavior during failure. Distributed computing is involved since there are various independently operating services, which communicate with each other, may be interrupted at different times, and must coordinate changes to business information. One of the most significant risks is the evaluation since the overly simplified simulation may not provide for the actual disconnections, concurrency, and users' behavior. Absence of real-world usage and failure data can lead to a result according to which some mechanism works in my tests and does not actually solve some organizational problem.

Connection to the Three Assigned Readings

"Types of Engineering Design" is a useful category to consider for classifying my project without presuming that I will have to create an innovative solution. Existing solutions may simply require a modification for the particular workflow, while software design should include aspects of components, interfaces, and requirements from the very beginning. This will help me avoid making novelty more significant than the problem itself.

"Research and Analysis of Existing Solutions" will affect my analysis by making me consider a reasonable comparison of the methods before applying any of them. Rather than just choosing a workflow engine or an outbox, I need to determine their assumptions and limitations and compare them in regard to particular failure scenarios.

In "Communications in Engineering Capstone Projects," the importance of identifying the need, making the assumptions clear, and identifying the risks involved is stressed. In the context of B05, this requires the specification of the projector complaint example, the identification of any unproven results as a hypothesis, and the presentation of the results at a later date.

References

[1] B05 — Resilient Case Management Services. Course opportunity catalog (title supplied; full published description and URL pending).

[2] Engineering Capstone Design, Project Planning, Organizing, and Executing. "Types of Engineering Design." Assigned course PDF.

[3] Engineering Capstone Design, Project Planning, Organizing, and Executing. "Research and Analysis of Existing Solutions." Assigned course PDF.

[4] Engineering Capstone Design, Project Planning, Organizing, and Executing. "Communications in Engineering Capstone Projects." Assigned course PDF.

[5] Microsoft Learn. "Create and manage basic queues for cases."

https://learn.microsoft.com/en-us/dynamics365/customer-service/administer/set-up-queues-manage-activities-cases

[6] Microsoft Learn. "Implement the Transactional Outbox pattern by using Azure Cosmos DB."

https://learn.microsoft.com/en-us/azure/architecture/databases/guide/transactional-out-box-cosmos

[7] Camunda 8 Documentation. "Job workers."

https://docs.camunda.io/docs/components/concepts/job-workers/